

Proof Buddy — Developer Guide

This guide is for developers (or AI systems) who need to understand the internal logic of Proof Buddy and make modifications to it. It explains where things live, how the pieces connect, and what to change when extending the system.

1. Where Core Logic Lives

The Proof Engine: [django_server/expression_tree/](#)

This is the heart of the system. All proof validation logic lives here. The most important files, in order of importance:

1. **ERProofEngine.py** — Orchestrates proof sessions. If you want to understand how a proof step is processed end-to-end, start here.
2. **ERRuleset.py** — Contains every proof rule. If you want to add a new rule, this is where it goes.
3. **ERCommon.py** — Contains **Node** (the AST node class) and **makeJson()** (converts a tree to the JSON the frontend uses). If the data structure of expressions needs to change, change it here.
4. **Parser.py** — Converts a raw string like `"(+ (* 2 n) 5)"` into a **Node** tree. Tokenization and recursive descent parsing are both here.
5. **expressionDefinition.py** — First-pass type assignment. Labels nodes with their initial types based on built-in function tables.
6. **Decorator.py** — Second-pass type propagation and type error detection.
7. **ERGenerics.py** — Represents generic (symbolic) variables with type constraints.
8. **ERobj.py** — Definitions of all built-in operators as **ERobj** descriptors with type signatures.
9. **IndProofs.py** — The **IndProof** class: wraps two **TwoSidedProof** objects to represent base case + leap step.

API Views: [equational_reasoning_api/views.py](#) and [induction_api/views.py](#)

These are the bridge between the web layer and the proof engine. Read them to understand what data is expected from the frontend and what is returned. They are relatively thin: their job is to (1) deserialize the request, (2) retrieve or create the in-memory proof object, (3) call the proof engine, (4) persist results to the database, and (5) return the result as JSON.

Frontend Pages: [client/src/pages/](#)

- **EquationalReasoningNew.js** — The active equational reasoning UI. ~400 lines. Drives the equational proof workflow.
- **InductionRacket.js** — The active induction UI. ~2400 lines. More complex because it handles base/leap cases and more proof metadata.

The Key React Component: [client/src/components/PersistentPad.js](#)

This component renders one proof line and handles node selection. It reads **jsonTree** (the flat JSON dictionary from the backend) to enable tree navigation with arrow keys. This is the component that is

replicated for each row in the proof editor.

2. Where Proof Rules Are Implemented

All rules are in `django_server/expression_tree/ERRuleset.py`.

Rule class hierarchy

```

Rule (abstract base)
├── BuiltIn          – Native Racket functions (cons, first, rest, null?, etc.)
│   └── If           – Special conditional rule with generic-aware branching
├── Math             – Algebraic simplification via SymPy
│   ├── Plus
│   ├── Minus
│   ├── Times
│   ├── Quotient
│   ├── Remainder
│   └── Expt
├── Axiom            – First-principles identities with parameter matching
│   ├── ConsProp     – (first (cons a L)) → a, (rest (cons a L)) → L
│   ├── FirstProp / RestProp
│   ├── NullQCons    – (null? (cons a L)) → #f
│   ├── ZeroQPlus    – (zero? (+ n 1)) → #f
│   ├── MinusPlus    – (- (+ n 1) 1) → n
│   └── AndProp      – (and #t X) → X, etc.
├── UDF              – User-defined recursive functions
└── IH               – Inductive Hypothesis (induction proofs only)

```

How a rule is registered and looked up

Each `ERProof` carries a `ruleSet` dictionary. This dictionary is populated when the proof is initialized in:

- `equational_reasoning_api/views.py` → `set_current_proof()`
- `induction_api/views.py` → `start_induction_proof()`

The rule set maps rule labels (strings like `"+"`, `"cons"`, `"length"`) to `Rule` instances. When a user types `"eval +"`, the label `"+"` is extracted and looked up in this dictionary to find the corresponding `Math.Plus` rule object.

3. How to Add a New Inference Rule

Step 1: Add the rule class to `ERRuleset.py`

Create a new class that inherits from the most appropriate base class (`Axiom`, `BuiltIn`, `Math`, or `Rule` directly). You must implement two abstract methods:

```
def isApplicable(self, node: Node) -> bool:
    """Return True if this rule can be applied to `node`."""
    ...

def insertSubstitution(self, node: Node) -> Node:
    """Return the new Node that should replace `node`."""
    ...
```

For an axiom-style rule (structural pattern matching): Subclass `Axiom`. Use `verifyStructure()` to check the node's shape, `matchParams()` to bind pattern variables, and `verifyValues()` to confirm semantic constraints on the matched values.

For a symbolic math rule: Subclass `Math`. Use `sympy.simplify()` on the `mathStr()` representation of the node.

For a built-in function evaluation: Subclass `BuiltIn`. The `insertSubstitution()` method should call the appropriate function from `ERobj.py` on the node's children and return the result wrapped in a new `Node`.

Step 2: Register the rule in the proof initialization

In `ERProofEngine.py`, inside the `ProofComponent.__init__()` method (or in the view that prepopulates the rule set), add your new rule to the `ruleSet` dictionary:

```
self.ruleSet["my-rule-label"] = MyNewRule(label="my-rule-label",
ruleType=RuleType.AXIOM)
```

This makes the rule available to users who type `"apply my-rule-label"` or `"eval my-rule-label"`.

Step 3: (Optional) Add to the frontend rule reference card

In `client/src/components/RuleSet.js`, add the new rule's name and syntax to the displayed reference list so students can see it.

Step 4: Add tests

Add test cases to `django_server/expression_tree/testApplyRule.py` or the relevant `tests.py` file. See [TESTING.md](#) for the full catalog. Run with:

```
cd django_server
python manage.py test equational_reasoning_api
python -m pytest expression_tree/testApplyRule.py
```

4. How to Add a New User-Defined Function (UDF)

UDFs are created in two ways:

1. **By users at runtime:** Through the `Definitions.jsx` component on the frontend. The definition is sent to the backend via `proofsService` and stored in the `proofs.Definition` database table.
2. **As default UDFs built into the system:** In `django_server/expression_tree/default_udfs.py`. Add a new entry to the `DEFAULT_UDFS` array with the fields `id` (negative integer), `label`, `def_type`, `expression`, `applied`, `is_default`, and `deletable`. Both `equational_reasoning_api` and `induction_api` views must call the `default_udfs` loading logic when initializing a proof.

How UDFs are processed

When a UDF is added to a proof:

1. `ProofComponent.addUDF(label, typeStr, body)` is called.
2. The body is parsed into a Node tree by `Parser.makeBasicAst()`.
3. The UDF is registered in the `ruleSet` as a UDF rule instance.
4. The `UDF.insertSubstitution()` method deep-copies the body tree and replaces parameter names with the actual argument subtrees passed at the call site.

5. How to Modify Proof Validation Behavior

Changing how a specific rule works

Modify the `isApplicable()` or `insertSubstitution()` method of the relevant rule class in `ERRuleset.py`.

Changing type checking

Modify `Decorator.py` (propagation rules) or `expressionDefinition.py` (initial type assignment). These two files together implement the type system.

Changing what constitutes a complete proof

Modify `TwoSidedProof.checkComplete()` in `ERProofEngine.py`. The conditions are:

- LHS last expression == RHS last expression (as strings)
- No blank lines except possibly the final trailing blank
- No lines with `hide_expression` or `hide_justification` set to True

Adding new generic variable constraints

Modify `ERGenerics.py`. Add new subclasses of `ERGeneric` if you need a constraint type that does not exist yet. Constraints are enforced during rule application when comparison operators are called on `GenericInt` instances.

Adding a new proof mode (e.g., a third proof structure)

1. Create a new Python class in `expression_tree/` extending `ProofComponent`.
2. Create a new Django app analogous to `equational_reasoning_api` or `induction_api`.
3. Add models, views, serializers, and URLs for the new mode.
4. Create a new React page component analogous to `EquationalReasoningNew.js`.
5. Add a new API service file in `client/src/services/`.

6. Register the route in `client/src/routes/ProofRoutes.js`.

6. How the Frontend Communicates with the Backend

API call pattern

Every API call follows this pattern:

```
// In a service file (e.g., equationalService.js):
export const applyRule = (payload) => {
  return axios.post(
    `${BASE_URL}/api/v1/equational/apply-rule`,
    payload,
    { headers: { Authorization: `Token ${getToken()}` } }
  );
};

// In the page component (EquationalReasoningNew.js):
const response = await equationalService.applyRule({
  side: 'LHS',
  currentRacket: '(+ (* 2 n) 5)',
  rule: 'eval +',
  startPosition: 3,
  selectedNode: 3,
  lineNumber: 2,
});
```

The `BASE_URL` is read from the `REACT_APP_BACKEND_API_BASE_URL` environment variable (set at build time in production, read from the dev proxy in development).

State management pattern

Each proof editor page maintains an array called `racketRuleFields` (managed by the `useRacketRuleFields` hook). Each element represents one line in the proof and holds:

- `racket`: The expression string (what's displayed)
- `rule`: The applied rule label
- `startPosition`: Node ID where the rule was applied
- `selectedNode`: The node the user has currently selected
- `jsonTree`: The flat JSON dictionary for that line's expression tree
- `resultNode`: The node ID that changed (for highlighting)
- `errors`: Array of error messages
- `hide_expression` / `hide_justification`: Visibility flags

The array always has a trailing blank element so there is always an empty line ready for the user's next step.

Authentication

The auth token is stored in a cookie (via `js-cookie`) on login. The `AuthProvider` context reads it and exposes it. Every service function reads it from the context or directly from the cookie to set the `Authorization` header.

7. How the Database Schema Relates to the Proof System

Key tables and their roles

Model	Table	Role in proof system
Account	accounts_account	User identity; <code>is_instructor</code> controls access to instructor features
Definition	proofs_definition	User-defined function (label, type string, body expression)
Generic	proofs_generic	Symbolic variable (label, type, constraints)
EquationalProof	equational_reasoning_api_equationalproof	Metadata for one equational proof session: name, LHS/RHS goals, current side, completion flag
EquationalProofLine	equational_reasoning_api_equationalproofline	One step in the proof: side (LHS/RHS), expression string, JSON tree, rule, start position, selected node, result node, errors, visibility flags
InductionProof	induction_api_inductionproof	Metadata for an induction proof: includes <code>proof_type</code> (int/list), induction variable, anchor value, leap variable, IH strings
InductionProofLine	induction_api_inductionproofline	Same as equational but with an added <code>case</code> field (base/leap)

Model	Table	Role in proof system
Course	assignments_course	A course section: instructor, students
Assignment	assignments_assignment	A problem set owned by a term with a due date
AssignmentProof	assignments_assignmentproof	A problem set owned by a term with a due date
StudentProofMapping	assignments_studentproofmapping	Storing of relationships between assignments and proofs students work on for the assignments.
CourseInvitation	assignments_courseinvitation	Tracker for invitations manually created by instructors to students for their courses.

Why there is a cache AND a database

The database stores a durable record of every proof line. The Django cache stores a live, in-memory `TwoSidedProof` (or `IndProof`) Python object. The cache exists because reconstructing the full proof object from the database on every request would be expensive — the Python expression trees are complex nested objects. The cache accelerates this by keeping the proof object hot. When a proof is loaded, all lines are fetched from the database and replayed into the in-memory object.

If the cache expires (30-minute timeout), the next request rebuilds the proof from the database. This means the database is the authoritative record and the cache is a performance layer.

8. Key Invariants to Preserve When Modifying Code

These are the "rules of the road" discovered through development and written in `AINotes.txt`:

1. **Proof line immutability:** Only the bound (current) line's expression tree changes when a rule is applied. All earlier lines must never be mutated.
2. **Tree, not string:** The backend works with expression trees (`Node` objects). Do not work with raw expression strings as a substitute for tree operations. The `makeJson()` → `findNode()` pipeline is the correct way to locate and manipulate nodes.
3. **Trailing blank line:** The `racketRuleFields` array always ends with a blank entry so there is always a ready slot for the user's next step. Guard against accidentally inserting a second blank when editing

mid-proof.

4. **SessionStorage is the client cache:** User definitions and generics are persisted in `sessionStorage` on the frontend to survive page re-renders. Do not clear it inadvertently. When merging backend-returned definitions with client-side state, preserve the `applied` flag from session storage.
5. **Induction and equational are separate pipelines:** They share the `expression_tree` library but have distinct API routes (`/api/v1/induction/` vs `/api/v1/equational/`), distinct database models, and distinct frontend pages. Never mix their service calls.
6. **IH is induction-only:** The `IH` rule type is only valid inside an `IndProof`. In the equational mode, submitting `"apply IH"` should return an error.
7. **startPosition from source, not destination:** When applying a substitution, the `startPosition` comes from the source (previous) line's selected node, not from the blank destination line.
8. **Premises from their own jsonTree:** Proof premises are rendered from their own stored `jsonTree`, not from a shared `jsonTreeRep`. Mixing these causes incorrect tree navigation state.